

BERT-based Book Genre Classifier

NUANS - Project Report

Antonello Giorgio
1836529

giorgio.1836529@studenti.uniroma1.it

1 Introduction

The aim of this project is to provide a system able to classify books into various genres based on their summaries. The model architecture leverages BERT (Bidirectional Encoder Representations from Transformers), which takes the "encoder" part of a typical transformer architecture and produces embeddings useful for text classification.

2 Related Work

Aniket Patra, Arnanta Chatterjee and Sirshendu Ganguly's project **Book Genre Prediction using Book Summary** aims at applying Machine Learning algorithms to categorize texts based on genre. They exploited a subset of the **CMU Book Summary Dataset**, which contains plot summaries extracted from Wikipedia. Since the dataset was unevenly distributed, they processed it to obtain a more uniform version, which contains **3000 book summaries** and has **6 main genres** - "Crime Fiction", "Fantasy", "Historical novel", "Horror", "Science Fiction", "Thriller". Regarding the Machine Learning techniques, they used **one-vs-rest logistic regression (OVR-LR)** and **support vector machine (SVM)**. For the experiments, they first tried to split the dataset in 80% Train - 20% Test samples, then they also tried 85% - 15%. For both splits, three different algorithms were performed: OVR-LR, SVM with Linear Kernel and SVM with RBF kernel. The **highest accuracy (79.55%)** was obtained during a **85% - 15% split** while using **SVM with RBF kernel**.

3 Methodology

3.1 Data Preprocessing

The project starts with loading data from a CSV file, corresponding to the processed dataset of Patra et al. project. The lengths of the summaries are

calculated to understand the data distribution (see Figure 1).

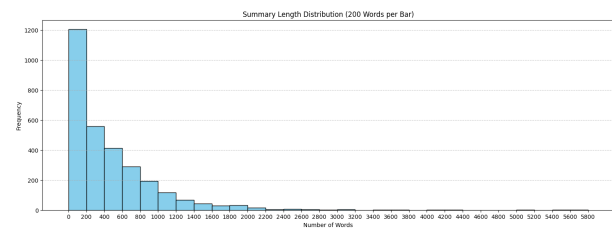


Figure 1: Plot summaries length (in words) distribution.

Then, for each token obtained by using *spaCy* on each summary:

- it is converted to the lowercased version.
- it is checked against a set of stop words provided by *spaCy*. If it's not a stop word:
 - If the token is a Named Entity, it's enclosed with "START:" and "END" markers along with its entity type
 - If it's not an NER, it is lemmatized to its base form and then stemmed to reduce it to its root form.

3.2 Tokenization

BERT tokenizer is employed to tokenize the book summaries, padding them to a maximum length determined at batch time and truncating if necessary. This prepares the data for input to the BERT model, that accepts only sequences no longer than 512 tokens: since the percentage of summaries with less than 512 words is equal to 66.83%, it's not a significant issue, although we could lose some important content for the longer summaries.

3.3 BERT Transformer Model

The core of the classifier is the **BERT transformer model**, which is fine-tuned for the book genre clas-

sification task. The BERT model captures contextual information from the text, taking as input the tokenized sequences corresponding to the plot summaries, accompanied by attention masks to ignore padding tokens. From the output, I decided to take the embedding of the first token from the last layer of the BERT model as an aggregate representation of the entire input sequence.

3.4 Dropout Layer

To prevent overfitting, a dropout layer is added to the model. With a dropout probability chosen at hyperparameters level, it randomly drops a certain proportion of neurons during training, encouraging the model to be more robust.

3.5 Classifier Layer

A linear classifier layer is used to make predictions based on the BERT model's output. This layer maps the output of the dropout layer to logits representing the score for each possible genre label for the plot summary. Finally, in the training phase, the logits are transformed into probabilities using the softmax function (integrated in the Cross-Entropy loss function) and the predicted genre for the current book will be obtained taking the label with the highest probability.

4 Experiments

The metric used to evaluate model performances is **validation accuracy**, although I also kept track of training accuracy and loss behavior for both training and validation sets. To optimize performance and minimize time and memory consumption, I implemented **mixed-precision training**, which involves using lower-precision (float16) data for model's forward and backward passes and higher-precision (float32) data for critical operations, like weight updates, to maintain numerical accuracy. Additionally, I used a **dynamic learning rate schedule** to make the model converge faster and stabilize training by starting with small learning rates and gradually increasing them. It's worth noting that **AdamW optimizer** was employed: it is a variant of the well-known Adam optimizer that incorporates weight decay regularization, particularly well-suited for the BERT transformer. I experimented with different hyperparameters configurations, and plots with confusion matrices are shown below:

- **Only BERT**, Batch size: 16, Number of

epochs: 20, Learning rate: 2e-05, Maximum sequence length: 512, Dropout probability: 0.1, Gradient accumulation steps: 2, Weight decay: 0.

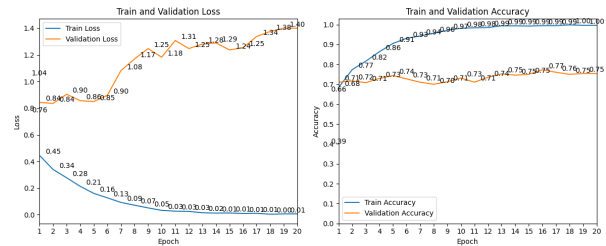


Figure 2: Validation Accuracy = 0.7729, reached at Epoch 17/20.

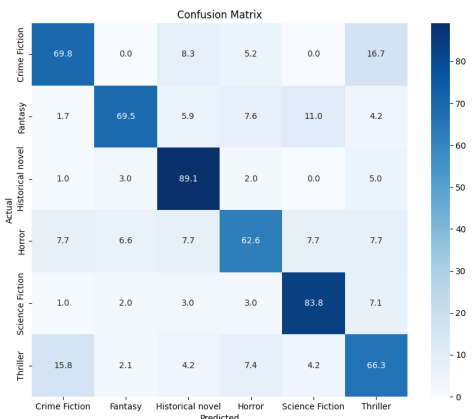


Figure 3: Confusion Matrix

- **Only BERT**, Batch size: 128, Number of epochs: 20, Learning rate: 1e-05, Maximum sequence length: 256, Dropout probability: 0.5, Gradient accumulation steps: 2, Weight decay: 0.01.

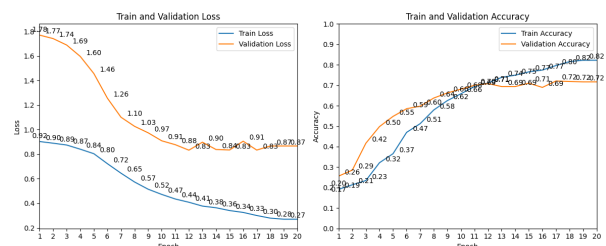


Figure 4: Validation Accuracy = 0.7208, reached at Epoch 18/20.

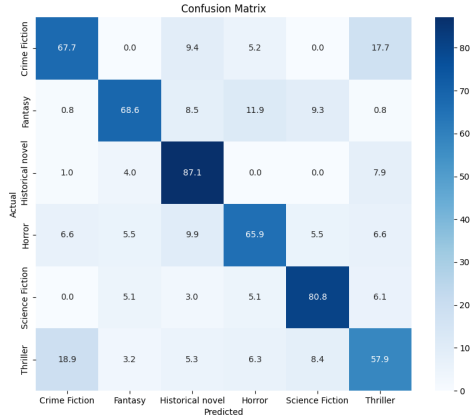


Figure 5: Confusion Matrix

- **BERT+stop words**, Batch size: 16, Number of epochs: 20, Learning rate: 2e-06, Maximum sequence length: 512, Dropout probability: 0.5, Gradient accumulation steps: 1, Weight decay: 0.1.

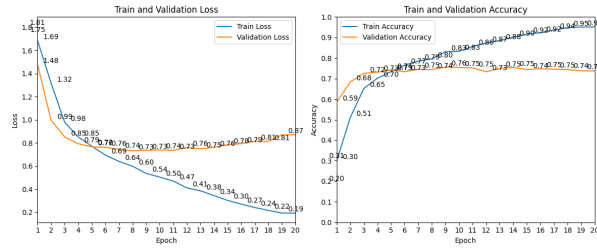


Figure 6: Validation Accuracy = 0.7562, reached at Epoch 10/20.

- **BERT+stop words+NER**, Batch size: 16, Number of epochs: 20, Learning rate: 2e-06, Maximum sequence length: 512, Dropout probability: 0.5, Gradient accumulation steps: 1, Weight decay: 0.1.

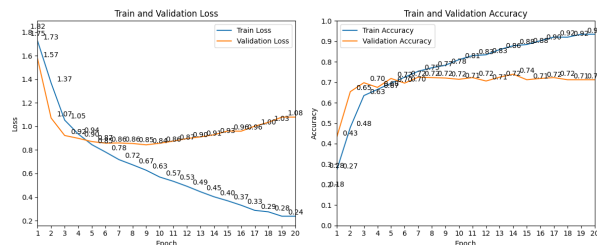


Figure 7: Validation Accuracy = 0.7396, reached at Epoch 15/20.

- **BERT+all the preprocessing**, Batch size: 16, Number of epochs: 20, Learning rate: 2e-

05, Maximum sequence length: 512, Dropout probability: 0.3, Gradient accumulation steps: 2, Weight decay: 0.1.

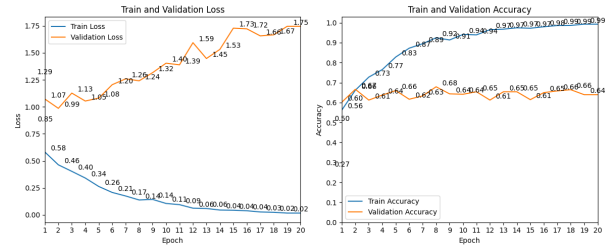


Figure 8: Validation Accuracy = 0.6792, reached at Epoch 9/20.

5 Conclusion

From the experiments, we can surely affirm that only BERT-based models successfully perform on text genre classification task, while using preprocessing techniques decreases the performances, indicating a more negative response towards a more processed representation of the summaries. Validation accuracy is always higher than 70%, reaching a peak of 77.29%. Having a lower maximum sequence length gives us a better loss/accuracy behavior during epochs, but a lower accuracy than the experiments with longer sequences: this may be due to a less amount of content to analyze for the prediction. From the confusion matrices, we can see that "Historical Novel" and "Science Fiction" books are more likely to be classified correctly w.r.t. other genres.

In comparison to the related work (section 2), since the accuracy reached with SVM using a RBF kernel is 79.55%, the BERT-based model still has to be improved with additional layers and/or hyperparameter tuning.